

Kernel Image Processing

Fabio Sucameli
Department of Information Engineering
University of Florence
`fabio.sucameli@edu.unifi.it`

Abstract

Kernel image processing is a fundamental technique in computer vision and graphics, where a small matrix of weights, called a kernel or filter, is systematically applied over an image through convolution. Each output pixel is computed as a weighted combination of its neighboring pixels, enabling operations such as blurring, sharpening, edge detection, embossing, and more. This method enhances specific features or textures within the image, playing a critical role in many applications including feature extraction, noise reduction, and artistic effects. The design and selection of different kernels directly determine the type of transformation applied to the image.

1. Introduction

This project investigates the transition from sequential to parallel image processing, aiming to demonstrate the performance improvements achieved through GPU acceleration. The implemented system loads a batch of images and applies a series of convolution filters, including box blur, Gaussian blur, sharpening, edge detection, and emboss effects. Initially, a purely sequential CPU-based approach was developed using C++ and OpenCV, serving as a baseline for performance comparison. The parallel CUDA-based version was then introduced to leverage GPU texture and constant memory features for faster execution. All GPU-based experiments were conducted on a personal computer equipped with an **NVIDIA GeForce RTX 4060**, enabling effective hardware acceleration of convolution operations.

The primary objective is to showcase how moving computationally intensive tasks like image convolution from CPU to GPU can drastically reduce execution times, especially when processing high-resolution images or large datasets.

2. Sequential Image Processing Pipeline

The sequential pipeline represents the initial CPU-bound implementation. It focuses on loading input images, applying different filters sequentially to each color channel, and saving the processed images. This design establishes a performance baseline before introducing parallel GPU acceleration.

2.1. Workflow Overview

The sequential processing follows these main steps:

- **Image Loading:** Reads images from an input directory using OpenCV.
- **Channel Splitting:** Splits each image into its three color channels (B, G, R).
- **Filter Application:** Applies various convolution filters separately to each channel using sequential CPU processing.
- **Image Merging:** Recombines the filtered channels into a final output image.
- **Image Saving:** Writes the processed images to an output directory.
- **Time Measurement:** Records the execution time for CPU filtering to allow later comparison with GPU-based processing.

2.2. Key Functions

- **applyFilterSequential(output, filter):** This method, implemented within the `ImageProcessor` class, performs convolution between the input image and the specified filter kernel. It follows a straightforward nested loop structure where each output pixel is computed by multiplying and summing neighboring pixels with corresponding kernel coefficients. Padding is added to handle border pixels.

- **createPaddedImage(padY, padX)**: To correctly apply convolution at the image borders, this function creates a padded version of the original image by extending its size and filling the new areas with zeros.
- **measureTime(func)**: A template function that measures and returns the execution time of any given function, facilitating performance profiling of CPU and GPU operations.

2.3. Filter Kernel Definitions

The `FilterKernel` class provides several predefined convolution kernels, each designed to apply a specific image transformation. Below are the details of each filter:

- **createIdentityFilter()**: Creates a 3x3 identity kernel, which leaves the image unchanged after convolution. The kernel matrix is:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

This operation is useful for testing and debugging the convolution pipeline.



Figure 1. Result after applying the Identity filter.

- **createBoxBlurFilter(size)**: Generates a normalized box blur filter of customizable odd size. In this project, a fixed **11x11** box blur is used. Each element has equal weight, summing up to 1. The general structure of a box blur kernel is:

$$\frac{1}{size^2} \begin{bmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{bmatrix}$$

It smooths the image by averaging neighboring pixels, effectively reducing noise and fine details.



Figure 2. Result after applying the Box Blur filter (11x11).

- **createSharpenFilter(factor)**: Generates a sharpening filter to enhance edges and fine details. The 3x3 kernel with sharpening factor λ is:

$$\begin{bmatrix} 0 & -\lambda & 0 \\ -\lambda & 4\lambda + 1 & -\lambda \\ 0 & -\lambda & 0 \end{bmatrix}$$

Higher values of λ lead to stronger edge enhancement.



Figure 3. Result after applying the Sharpen filter.

- **createGaussianFilter(size, sigma)**: Creates a Gaussian blur kernel based on the 2D Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

In this project, a Gaussian filter of size **15x15** with standard deviation $\sigma = 5.0$ is used. Gaussian blur provides a more natural smoothing effect compared to box blur, giving greater weight to central pixels and progressively less to distant ones.



Figure 4. Result after applying the Gaussian filter (15x15).

- **createEdgeFilter()**: Constructs a Laplacian-based kernel for edge detection, emphasizing areas with rapid intensity changes:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This filter highlights contours and borders in the image.



Figure 5. Result after applying the Edge Detection filter.

- **createEmbossFilter()**: Creates a kernel that produces an emboss effect, simulating a three-dimensional appearance by highlighting edges in a specific direction:

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

It combines edge detection and shading to give a raised impression to the image.

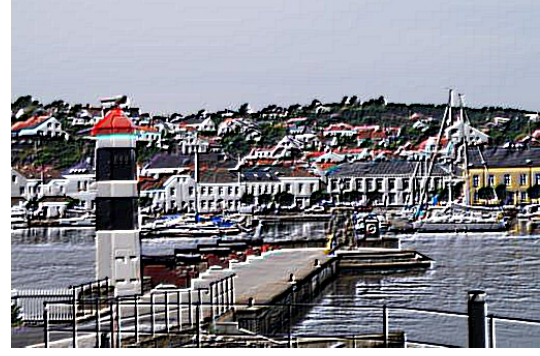


Figure 6. Result after applying the Emboss filter.

2.4. Convolution Formula

The sequential convolution operation applies the following formula for each pixel:

$$I_{out}(x, y) = \sum_{i=-r}^r \sum_{j=-r}^r K(i, j) \cdot I_{pad}(x + i, y + j)$$

where:

- $I_{out}(x, y)$ is the output pixel value.
- $K(i, j)$ represents the convolution kernel value at offset (i, j) .
- $I_{pad}(x + i, y + j)$ is the corresponding pixel value in the padded input image.
- r is the kernel radius (half the kernel size).

2.5. Performance Characteristics

The sequential CPU implementation, while straightforward and easy to implement, suffers from several limitations:

- **High Computational Cost:** Convolution scales linearly with the number of pixels and the kernel size, resulting in long processing times.
- **Low CPU Utilization:** The single-threaded design underutilizes the capabilities of modern multi-core processors.
- **Poor Scalability:** Processing time increases rapidly with higher image resolutions or larger datasets, making the approach impractical for real-time or large-scale applications.

Due to these performance limitations, it became necessary to explore parallelization strategies. From this point onward, the image processing pipeline was adapted to leverage GPU acceleration through CUDA, allowing for significant improvements in execution time and scalability.

3. Parallel Image Processing with CUDA

After analyzing the limitations of the sequential CPU implementation, GPU acceleration using CUDA was introduced to optimize the image processing pipeline. Different memory strategies and optimizations were explored to maximize performance and scalability.

3.1. Initial Approach: Global and Shared Memory

The first attempt to parallelize the convolution operations on GPU combined the use of **global memory** and **shared memory**. Each thread block loaded a tile from global memory into shared memory to accelerate local access during convolution. However, the approach remained inefficient: global memory access suffered from high latency, and shared memory introduced synchronization overhead and bank conflicts. Furthermore, the manual handling of image borders increased kernel complexity, and the large memory footprint for big convolution windows reduced GPU occupancy. Overall, this strategy failed to fully exploit the memory optimizations available for read-only image data.

3.2. Final Approach: Constant and Texture Memory

The final and most effective approach combined the use of **constant memory** for storing the convolution kernels and **texture memory** for reading the input image. This strategy allowed for faster global memory accesses, better memory caching, and automatic boundary handling.

Constant memory was used to store small convolution kernels (e.g., 15x15 for Gaussian blur), exploiting its fast cache for all threads. Texture memory, with its hardware-level spatial caching and optimized addressing modes, was employed for reading pixel data, greatly improving memory throughput especially when accessing neighboring pixels.

3.3. Optimization Techniques

Several CUDA-specific optimizations were implemented to maximize performance:

- **Block Size (BLOCK_SIZE = 16):** After experimenting with different block sizes (8, 16, 32), a value of **16** was selected as it provided the best balance between occupancy and memory usage. Larger blocks (32) caused excessive shared memory pressure, while smaller blocks (8) underutilized the device.
- **Constant Memory:** The convolution kernel was stored in `__constant__` memory to allow fast, read-only access for all threads without manual memory management.
- **Texture Memory:** The input image was bound to a CUDA texture object, enabling efficient 2D memory

access patterns and automatic spatial caching. This greatly accelerates convolution operations, especially when neighboring pixels are accessed repeatedly. Texture memory is optimized for spatial coherence and provides fast, cached reads across threads.

- **Pinned Memory:** Host memory for images was allocated using `cudaMallocHost()`, allowing faster memory transfers (host-to-device and device-to-host) through direct memory access (DMA).
- **Structure of Arrays (SoA):** Instead of interleaved RGB data, each color channel was stored separately, improving memory coalescing and cache efficiency during access patterns.
- **Streams:** Multiple CUDA streams were used to overlap memory transfers and kernel executions, reducing idle times and improving GPU utilization.
- **Exclusive Scan Optimization:** To manage dynamic memory allocation and indexing operations within kernels, an optimized parallel prefix sum (scan) operation was introduced, further reducing overhead.
- **Padding with Texture Addressing:** Instead of manually handling border conditions, `cudaAddressModeClamp` was used with texture memory to simulate zero-padding automatically when accessing out-of-bound coordinates.
- **`__restrict__` Keyword:** Pointers passed to device functions were annotated with `__restrict__` to inform the compiler that memory accesses would not alias, enabling better instruction-level optimization and memory access scheduling.

3.4. Performance Results

Thanks to these optimizations, a significant performance improvement was observed. Specifically, when applying a 15x15 Gaussian filter on a high-resolution image of size **3532x5456 pixels**, the speedup improved dramatically:

Initial speedup $\approx 13.021\times$

Final speedup $\approx 75.469\times$

This demonstrates the importance of carefully optimizing both memory access patterns and computational strategies in GPU programming.

3.5. CUDA Functions and Kernel Implementation

The final GPU implementation relied on a combination of utility macros, a custom CUDA kernel, and a structured memory management system to achieve optimal performance. The key components are described below:

- **CUDA.CHECK(call)**: A macro designed to simplify CUDA error checking. It wraps any CUDA API call and automatically checks for errors. In case of an error, it prints the error message, resets the device, and exits the function by returning `false`. This ensures robust execution and easier debugging without cluttering the code with manual error handling.
- **__constant__ float d_kernelConst[225]**: A constant memory array on the device, capable of storing convolution kernels up to size 15x15. The constant memory offers faster access and efficient broadcasting when all threads access the same filter coefficients during convolution, minimizing global memory traffic.
- **filterWithTexture** (CUDA Kernel): A device kernel that performs 2D convolution using texture memory for input access and constant memory for kernel coefficients. Each thread computes one output pixel by reading its local neighborhood via the `tex2D` function, which accesses a texture-bound image, and multiplying these values with the corresponding entries in the constant kernel. The use of `__restrict__` on pointers helps the compiler optimize memory access further by assuming no aliasing.
- **processImageCUDA** (Host Function): The main GPU processing pipeline. It prepares the data and coordinates the full image convolution through the following steps:
 - **Kernel Setup**: Copies the convolution kernel from host memory to constant memory using `cudaMemcpyToSymbol`.
 - **Pinned Memory Allocation**: Allocates host-pinned memory for both input and output buffers to accelerate memory transfers.
 - **Texture Binding**: Converts input images to CUDA arrays, creates texture objects with `cudaAddressModeClamp` addressing, and binds them for efficient, cached read-only access.
 - **Kernel Launch**: Launches the `filterWithTexture` kernel asynchronously in independent CUDA streams, one for each color channel (B, G, R).
 - **Asynchronous Memory Transfers**: Uses `cudaMemcpyAsync` and streams to overlap computation and memory transfers, improving throughput.
 - **Result Reconstruction**: Copies the output from device to pinned host memory, clamps values between 0 and 255, and reconstructs OpenCV grayscale images.

- **Resource Cleanup**: Properly releases all CUDA resources (texture objects, arrays, device memory, pinned memory, streams) to prevent memory leaks.

4. Introduction to Performance Results

The evaluation of the image processing pipeline involves analyzing the improvements achieved through parallelization. This section introduces the key performance metrics, including the calculation of speedup and the graphical representation of performance scaling with respect to the number of processes.

5. Performance Analysis: CPU vs GPU Speedup

To quantitatively evaluate the benefits of GPU parallelization, an experimental analysis was performed by comparing the execution times of the sequential CPU implementation and the parallel CUDA implementation on the same input image. The objective was to measure how the speedup varies depending on the computational complexity of the applied filter.

Three representative filters were selected for this analysis:

- **Identity Filter (3x3)**: Minimal computational load, used as a baseline case.
- **Box Blur Filter (11x11)**: Moderate computational load, with larger neighborhood operations.
- **Gaussian Blur Filter (15x15)**: High computational load due to a large kernel size and floating-point operations.

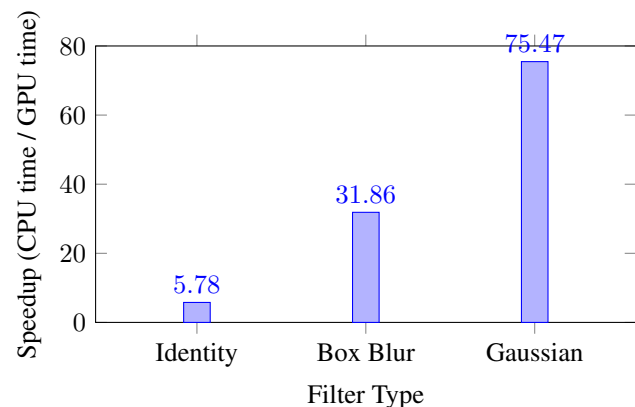


Figure 7. GPU Speedup over CPU for different filter complexities.

The results show that the speedup achieved by the GPU over the CPU increases significantly with the complexity

and size of the filter. While for a simple identity filter the speedup remains moderate, it grows considerably for larger convolution windows, reaching up to more than 75x for the Gaussian blur filter.

As illustrated in Figure 7, the performance advantage of GPU parallelization becomes increasingly evident as the filter size grows. This is due to the fact that larger convolution kernels require more operations per pixel, making the highly parallel architecture of GPUs substantially more efficient compared to the sequential CPU approach.

6. Speedup Analysis on Different Image Sizes

To further investigate the scalability of GPU parallelization, a second experiment was conducted by processing square images of increasing size, ranging from 512×512 to 8192×8192 pixels. The objective was to observe how the speedup varies with the input size for different convolution filters.

The analysis focuses on three representative filters:

- **Identity Filter (3x3)**
- **Box Blur Filter (11x11)**
- **Gaussian Blur Filter (15x15)**

Each point in the following graph represents the speedup obtained for a given image size and filter type.

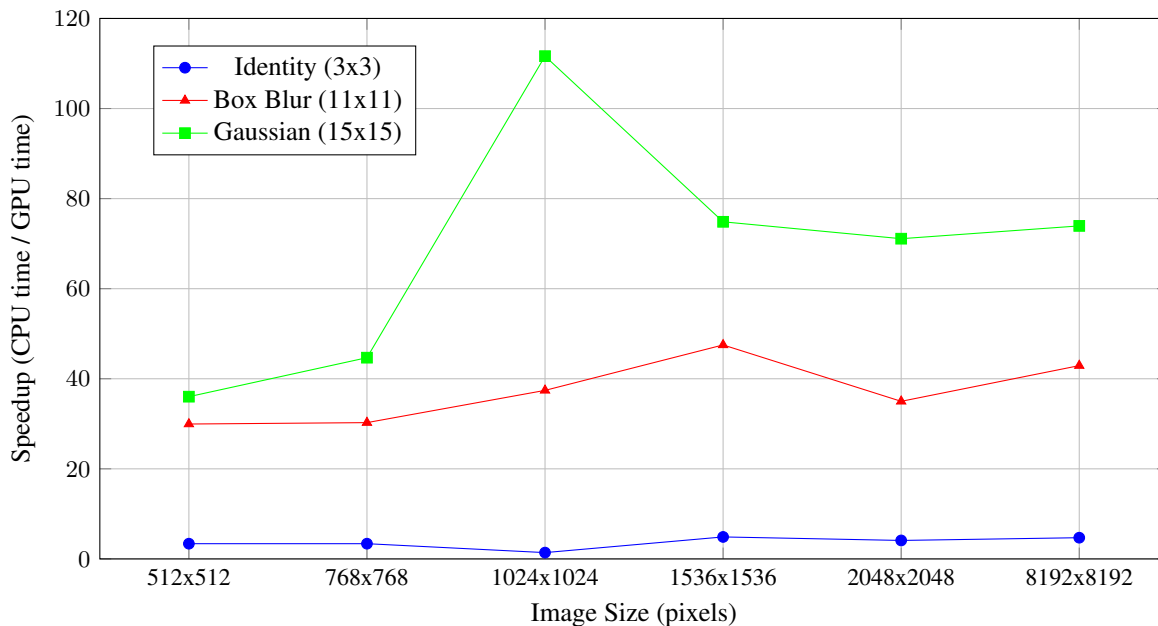


Figure 8. Speedup comparison for different image sizes and convolution filters.

As shown in Figure 8, the speedup generally increases with image size, particularly for computationally intensive

filters such as box blur and Gaussian blur. The Gaussian blur, in particular, shows the highest speedup, reaching over 100x in the 1024×1024 case and maintaining excellent scaling across larger images. In contrast, the identity filter achieves modest speedup values, indicating that when computational intensity is low, memory transfer costs dominate the overall execution time. Overall, the results highlight the strong scalability of GPU acceleration for complex, computation-heavy image processing tasks.

7. Conclusion

This project explored the transition from a sequential CPU-based image processing pipeline to an optimized parallel GPU implementation using CUDA. Convolution operations were initially implemented sequentially in C++ as a baseline for performance comparison. However, the high computational cost and poor scalability of the CPU approach made it necessary to adopt GPU parallelization.

The final CUDA implementation, leveraging memory hierarchies and optimization strategies, achieved substantial improvements in execution time and scalability. Overall, the project demonstrates how GPU acceleration enables efficient processing of large-scale, computation-heavy tasks, making real-time performance feasible for applications such as deep learning, computer vision, and digital image processing.